Imperial College
London

Lecture 8

Chip Testing

Prof Peter YK Cheung

Department of Electrical & Electronic Engineering

URL: www.ee.ic.ac.uk/pcheung/teaching/EE4-VLSI/
E-mail: p.cheung@imperial.ac.uk

This lecture is all about how VLSI testing is conducted. It also covers fault models and some basic algorithms for test vector generation.

Important of Testing

- Testing is one of the most expensive parts of chips
  - Logic verification accounts for > 50% of design effort for many chips
  - Debug time after fabrication has enormous opportunity cost
  - Shipping defective parts can sink a company
- Example: Intel FDIV bug
  - Logic error not caught until > 1M units shipped
  - Recall cost $450M (!!!)

VLSI Tests

1. Functional test
2. Silicon debug
3. Manufacturing tests

Stage when defect found	Cost per device
Wafer	$0.01 - $0.1
Packaged chip	$0.1 - $1
Board	$1 - $10
System	$10 - $100
Field	$100 - $1000

W&H p659-660

Tests fall into three main categories.

a) **Functionality test** or logic verification: Verifies that the chip performs its intended function. These tests are before tapeout to verify the functionality of the circuit.

b) **Silicon debug**: The second set of tests run on the first batch of chips that return from fabrication. These tests confirm that the chip operates as it was intended and help debug any discrepancies. They can be much more extensive than the logic verification steps because the chip can be tested at full speed in a system. This silicon debug requires creative detective work to locate the cause of failures because the designer has much less visibility into the fabricated chip compared to during design verification

c) **Manufacturing tests**: Third set of tests. Verifies that every transistor, gate and storage element in the chip functions correctly. These tests are conducted on each manufactured chip before shipping to the customer to verify that the silicon is completely intact.

In some cases, the same tests can be used for all three steps, but often it is easier to use one set of tests to chase down logic bugs and another separate set optimized to catch manufacturing defects.

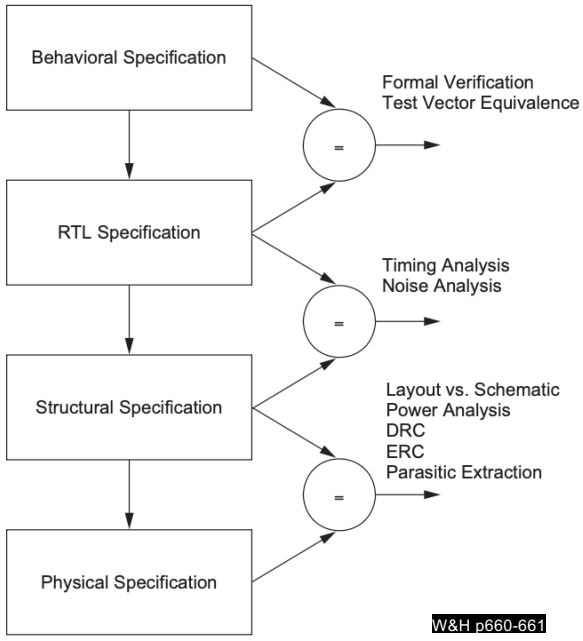
Because of the complexity of the manufacturing process not all die on a wafer function correctly (yield). Dust particles and small imperfections in starting material or photomasking can result in bridged connections or missing features. These imperfections result in what is termed a fault. The goal of the manufacturing test procedure is to determine which die are good and should be shipped to customers.

By detecting a malfunctioning chip early the manufacturing cost can be kept low (wafer- cheapest -> packaged chip -> board -> system -> field most expensive).

In an extreme example Intel failed to correct a logic bug in the Pentium floating point divider until more than 4 million units had shipped in 1994. IBM halted sales of Pentium based computers. Intel was forced to recall the flawed chips. The mistake and lack of prompt response cost the company an estimated US$450 million.

Functional/Logic Verification

- Does the chip simulate correctly?
  - Usually done at HDL level
  - Verification engineers write test bench for HDL
    - Can't test all cases
    - Look for corner cases
    - Try to break logic design
- Ex: 32-bit adder
  - Test all combinations of corner cases as inputs:
    - 0, 1, 2, 2^{31}-1, -1, -2^{31}, a few random numbers
- Good tests require ingenuity and deep understanding of the circuit



Verification tests are usually the first ones a designer might construct as part of the design process.

Often designers produce a **golden model** in one language (C, VHDL, SystemVerilog) and it becomes the reference against which all other representations are checked (e.g. RTL). Functional equivalence involves running a simulator at some level on the two descriptions of the chip (gate vs functional) and ensuring that the outputs are equivalent at some convenient check points in time for all inputs applied. This is most conveniently done in an HDL by employing a test bench i.e. a wrapper that surrounds a module and provides for stimulus and automated checking. The most detailed check might be on a cycle-by-cycle basis. Increasingly verification involves real time or near real time emulation in an FPGA based system to confirm system level performance. Recommended because of the increased complexity of chips and systems (e.g. WLAN chips – impossible to simulate unreliable channel, and interference).

Functional equivalence through various levels of the design hierarchy (behavioral, RTL, structural/logic, physical/transistors).

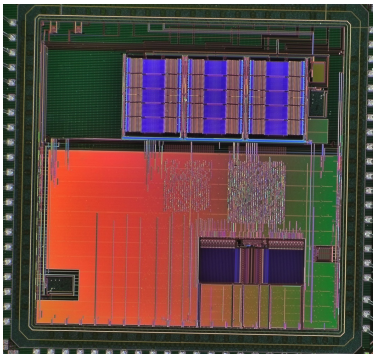
The best advice with respect to writing functional tests is to simulate as closely as possible the way in which the chip or system will be used in the real world. Often this is impractical due to slow simulation times and extremely long verification sequences. One approach is to move up the simulation hierarchy as modules become verified at lower levels.

Verification at the top chip level using an FPGA emulator offers several advantages over simulation. The emulation times can be near real time. Furthermore, the actual analog signals (if used) can be interfaced to the chip. Allows better assessment of system performance.

In most projects the amount of verification effort greatly exceeds the design effort.

Silicon Debug

- Test the first chips back from fabrication
  - If you are lucky, they work the first time
  - If not…
- Logic bugs vs. electrical failures
  - Most chip failures are logic bugs from inadequate simulation
  - Some are electrical failures
    - Crosstalk
    - Dynamic nodes: leakage, charge sharing
    - Ratio failures
  - A few are tool or methodology failures (e.g. DRC)
- Fix the bugs and fabricate a corrected chip



W&H p662-664

Basic digital debug hints

First tests on fabricated chip run in a lab environment. A circuit board is required including power for the IC, real world signal connections, clock inputs, digital interface to a PC.

Software to interface with the chip (through a serial UART port or other interface) – the lower-level software should provide for reading and writing registers in the chip. Alternatively provide interfaces for logic analyzer. These are easily added to a PCB design in the form of multi-pinned headers.

First functionality to be checked: registers (reads and writes)

When the chip has BIST you can run the commercial software that provides for this functionality over a boundary scan interface. This type of system automatically runs a set of tests on the chip that completely verify the correct operation of all gates and registers as defined by the original RTL description. If such a test interface was not used, you should pursue a manual effort in which the functionality of the chip is checked from the bottom up (not top down).

If anomalous behavior is detected debugging must start. The basic method is to postulate a method of failure, then test the hypothesis. Debug is an art itself. When postulating the cause of the bug and a test do one change at a time and observed the result.

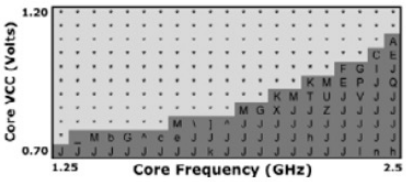
Silicon Debug principles

The major challenge in silicon debugging is when the chip operates incorrectly but you cannot ascertain the cause by making measurements at the chip pins or scan chain outputs. Number of techniques for directly accessing the silicon. First, specific signals can be brought to the top of the chip as probe points (small squares at the top metal layer that connect to the key points in the circuit that the designer has had the foresight to include before debug. The die can also be probed electrically or optically if mechanical contact is not feasible (electron beam probing, laser voltage probing, IR imaging, focused ion beam – this one can be used to cut wires or lay new conductors down).

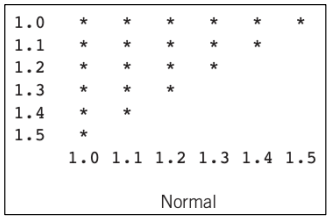
Manufacturing failures occur when a chip has a defect or is outside of the parametric specifications. Debug can reject chips with such problems. Functional failures are logic bugs or physical design errors that cause chip to fail under all conditions. Electrical failures occur when the chip is logically correct but malfunctions under certain conditions such as voltage, temperature or frequency.

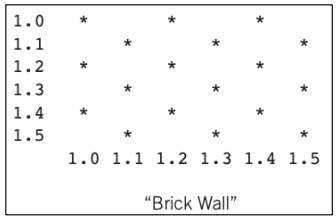
Shmoo Plots

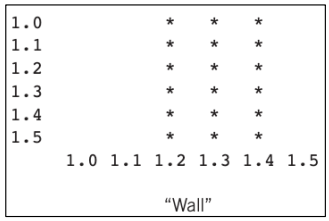
- How to diagnose failures?
  - Hard to access chips
    - Picoprobes
    - Electron beam
    - Laser voltage probing
    - Built-in self-test
- Shmoo plots
  - Vary voltage, frequency
  - Look for cause of electrical failures

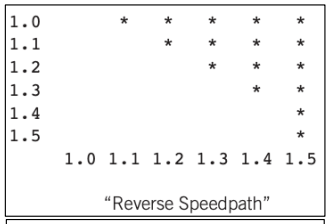


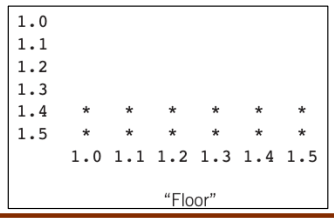
W&H p675-676

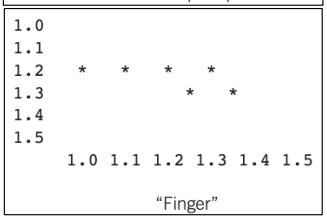












Shmoo plots can help to debug electrical failures in silicon. A Shmoo plot is often made with voltage on the Y-axis and clock frequency as the X-axis (or reversed). The test vectors are applied at each combination of voltage and clock speed, and the success of the test is recorded (as green). Often only a set of vectors applicable to a particular module is applied to diagnose a problem in that module. The Shmoo plots in this slide show a variety of conditions.

A **healthy normal** chip should operate at increasing frequency as the voltage increases. The **brick wall pattern** suggests that the chip may be randomly initialized in one of two states only one of which is correct. **The wall pattern** in which the chip fails to operate at any frequency above or below a particular voltage can indicate charge sharing, coupling noise, or a race condition. The **reverse speed path** behaviour indicates a leakage problem in which a weakly held node leaks to an invalid level before the end of the cycle. At higher voltage, the leakage is exacerbated and appears at shorter clock periods. **The floor** is a variant on the leakage problem where the part fails at low frequency independent of the voltage. A **finger** indicates coupling problems dependent on the alignment of the aggressor and victim where at certain frequencies the alignment always causes a failure.

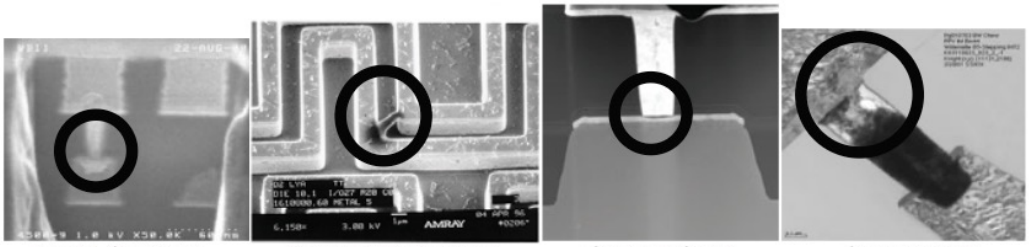
Manufacturing Test

- A speck of dust on a wafer is sufficient to kill chip
- *Yield* of any chip is < 100%
  - Must test chips after manufacturing before delivery to customers and only ship good parts
- Manufacturing testers are very expensive
  - Minimize time on tester
  - Careful selection of *test vectors*

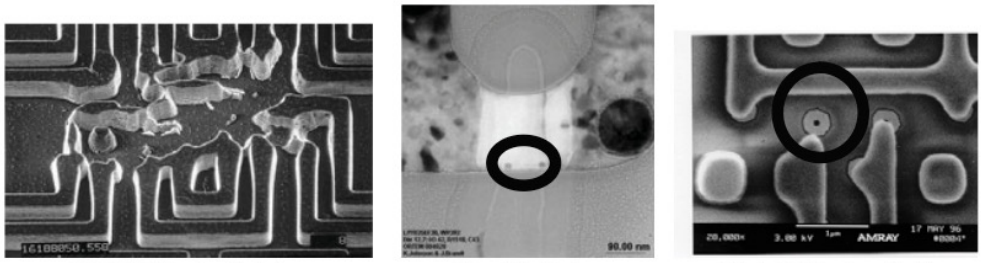
W&H p666

General purpose production tester is to used for manufacturing testing. Production testers are usually expensive pieces of equipment with configurable I/O ports (drive current, output levels, input levels) and huge amounts of RAM behind each test pin. The tester drives input pins from this memory on a cycle-by-cycle basis and samples and stores the levels on output pins. The figure shows a typical production tester. In the background you van see the four bay cabinet holding the drive electronics. To the right in the background is the controlling workstation. The test head is shown on the front center. This is where the chip is placed in the load board to be tested. The probe card or load board for the design under test is connected to the tester.

SEM images of manufacturing defects



Metal1 Shelving | Metal5 Film Particle (bridging defect) | Open Defects | Spongy via2 (infant mortality)



Metal5 Blocked Etch (patterning defect) | Spot Defects | Metal1 Missing Pattern (open at contact)

W&H p676-677

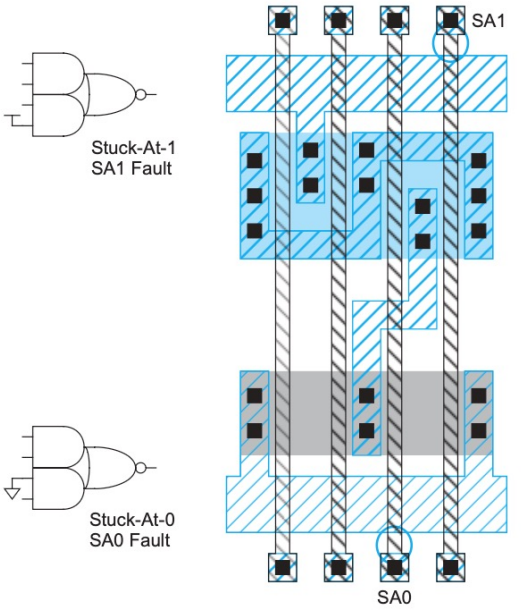
Manufacturing test is to identify defective parts in a wafer before they are packaged or shipped to customers. The typical target for defective rate is 350-1000 defects per million (DPM) chips shipped.

The slide above (from Intel and W&H) shows the different manufacturing defects that may occur during manufacture, which must be identified before shipping.

In designing a VLSI chip, such defects MUST be found via the testing regime adopted. Design-for-test is a philosophy in VLSI design where testing is considered concurrently with architectural exploration and detail design process.

Fault Modeling

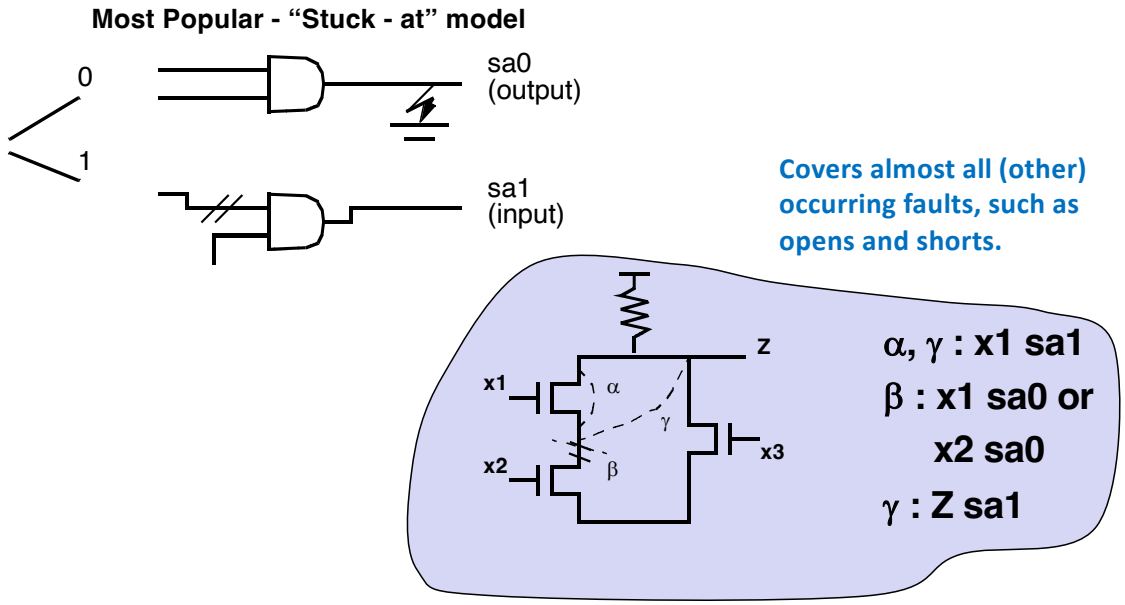
- Manufacturing faults can be due to shorts between two nodes or opens in a circuit in a wire
- This can cause very complicated behaviour
- Stuck-At model:
  - Assume all failures cause nodes to be “stuck-at” 0 or 1, i.e. shorted to GND or VDD
  - Not quite true, but works well in practice



Manufacturing faults can be of a wide variety and manifest themselves as short circuits between signals, short circuits to the supply rails and floating nodes. To evaluate the effectiveness of a test approach and the concept of a good or bad circuit we must relate these faults to the circuit model or in other words derive a fault model.

Most popular approach is the stuck-at model. Most tools consider only the short circuits to GND and Vdd – stuck at 0 (sa0), stuck at 1 (sa1). It does not cover stuck-at-open and stuck-at-short faults between two wires. If these are also introduced the test pattern generation process becomes complicated. Moreover, a large number of these faults are covered by the sa0-sa1 model. Therefore, the stuck-at model, while simple, works well in practice.

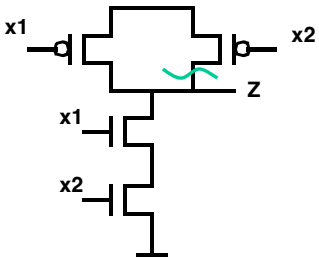
Stuck-at model covering other faults



Most CAD test tools consider only the short circuits to GND and Vdd – stuck at 0 (sa0), stuck at 1 (sa1). They do not cover **stuck-at-open** and **stuck-at-short** faults. If these are also introduced the test pattern generation process becomes complicated. Moreover, a large number of these faults are covered by the sao-sa1 model.

Consider the circuit on the bottom right: Resistive load MOS gate. Stuck-at-open (beta) and stuck at short (alpha, gama) faults. It can be observed that these faults are already covered by the sa0 and sa1 faults on the various nodes as shown above.

Limitation of stuck-at model: CMOS open fault



x1	x2	Z
o	x	1
1	1	0
1	0	Z_{n-1}

Sequential effect

Needs two vectors to ensure detection!

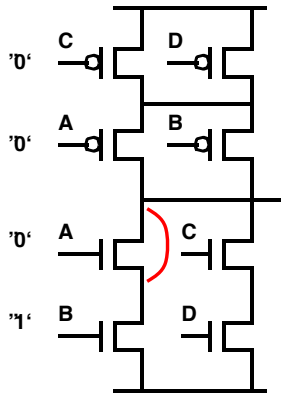
Other options: use stuck-open or stuck-short models

This requires fault-simulation and analysis at the switch or transistor level - Very expensive!

Shorts and open circuit faults can cause some interesting artifacts to occur in CMOS circuits that are not covered by the sa0, sa1 model.

Consider the two input NAND gate where a stuck at open fault has occurred. The truth table of the faulty circuit is shown. For the combination X1, X2 = 1,0 the o/p node is floating and retains its previous value while the correct value should be equal to 1. Depending upon the previous excitation vector this fault may or may not be detected. In fact, the circuit behaves as a sequential network. To detect this fault two vectors must be applied in sequence. The first one forces the o/p to 0 (1, 1) while the second applies the 1, 0 pattern

Limitation of stuck-at model: CMOS short fault



Causes short circuit between Vdd and GND for A=C=0, B=1

Possible approach:
Supply Current Measurement (IDDQ)
but: not applicable for gigascale integration

Also stuck-at short faults are troublesome in CMOS circuits since they can cause dc currents to flow between the supply rails for certain input values which produces undefined output voltages. I_{DDQ} test measures the change in quiescent current of a CMOS circuit due to short circuits.

Stuck at 0 or 1 model is is not perfect but it is easy to use and has large coverage of the fault space which made them the de facto standard model. It is often supplemented with other techniques such as functional test, IDDQ test, and delay test.

No single test is model is completely foolproof and a combination of several testing methods is often required.

Four concepts in testing

In VLSI tests, four concepts are important:

- **Controllability**: measures the ease of setting a node to 0 or 1 state
- **Observability**: measures the degree to which a node's state can be observed
- **Repeatability**: the ability to produce the same outut given the same input
- **Survivability**: the ability to continue functioning after a fault has occurred.

Controllability measures the ease of bringing a circuit node to a given condition using only the input pins. A node is easily controllable if it can be brought to any condition with only a single input vector. A circuit or node with low controllability needs a long sequence of vectors to be brought to a desired state. It should be clear that a high degree of controllability is desirable in testable designs.

Observability measures the ease of observing the value of a node at the output pins. A node with high observability can be monitored directly on the output pins. A node with low observability needs number of cycles before its state appears on the outputs. Given the complexity of a circuit and the limited number of output pins a testable circuit should have a high observability. This is the purpose of the testing techniques discussed next.

Repeatability is the ability to produce the outputs given the same input in a VLSI chip. Combinational logic and synchronous sequential circuits are always repeatable. Asynchronous circuits and intermittent faults are nondeterministic and therefore often are non-repeatable. That's why we rarely use asynchronous circuits in our designs. They are a nightmare to test!

Survivability in a system is the ability of it to function even when one or more faults occur. This normally involve either error correcting codes or redundancy.

How to test circuits?

- **Combinational Circuits**:
  controllable and observable - relatively easy to determine test patterns
- **Sequential Circuits**: State!
  Turn into combinational circuits or use self-test
- **Memory**: requires complex patterns
  Use self-test

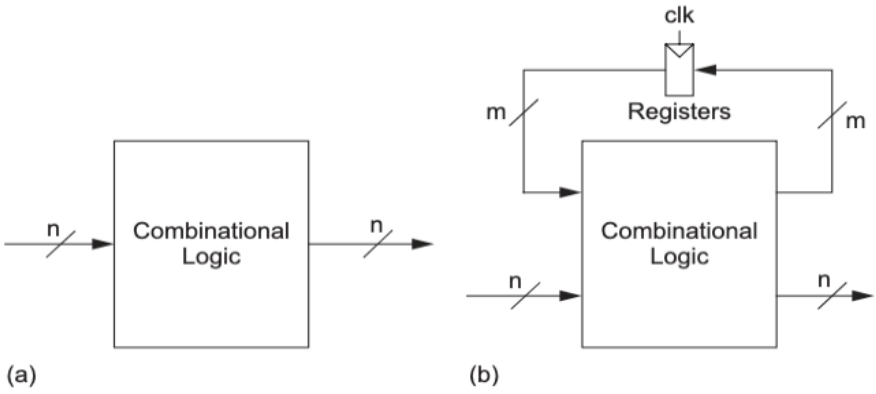
Combinational circuits are easily observable and controllable, and are therefore generally easy to test.

Synchronous sequential circuits are relatively easy to test as long as the inputs are **controllable** and the outputs are **observable**. However, a complicated synchronous circuit such as a complex finite state machine can be very difficult test fully, often requiring many cycles to enter desired state. However, with knowledge of how a particular FSM works and access to the state diagram, one an usually derive a test with relatively small number of clock cycle.

Testing memory requires to use complex data pattern and is usually test through some built-in self-test methods as will be discussed later.

Logic Verification Principles

- Scalability challenge in sequential circuit test.
- N = 25, M = 50.
- Exhaustive test needs around 3.8 x 10^{22} test vectors.
- Assume perform 1 test every microsecond, take 10^9 years!



W&H p670

Figure (a) shows a combinational circuit with N inputs. To test this circuit exhaustively, a sequence of 2N inputs (or test vectors) must be applied and observed to fully exercise the circuit. This combinational circuit is converted to a sequential circuit with addition of M registers, as shown in Figure (b). The state of the circuit is determined by the inputs and the previous state. A minimum of 2^{N+M} test vectors must be applied to exhaustively test the circuit.

Assume N = 25 and M = 50, or 2^{75} patterns, which is approximately 3.8×10^{22}. Assuming one had the patterns and applied them at an application rate of 1 μs per pattern, the test time would be over a billion years (10^9)!

Generating and Validating Test-Vectors

- **Automatic test-pattern generation (ATPG)**
  - for given fault, determine excitation vector (called **test vector**) that will propagate error to primary (observable) output
  - majority of available tools: combinational networks only
  - sequential ATPG is far more complex – normally require extra built-in test circuits
  - CAD tools (e.g. Cadence's Modus tools) are used to automatically produce test vectors to detect stuck-at faults in circuit (see later).
- **Fault simulation**
  - determines **fault coverage** of proposed test-vector set
    - ➢ Fault coverage = no. of faults detectable by test vector set/ total no. of faults
  - world-class quality test coverage >98.5% fault coverage
  - simulates correct network in parallel with faulty networks
- Both require adequate models of faults in CMOS integrated circuits

Complex task of determining what patterns should be applied so that a good fault coverage is obtained. This process was extremely problematic in the past when the test engineer - a different person than the designer – had to construct the test vectors after the design was completed. This required a substantial amount of wasteful reverse engineering that could have been avoided if testing had been considered early in the design flow. An increased sensitivity to design for testability and the emergence of **ATPG**.

ATPG determines minimum set of excitation vectors that cover a sufficient portion of the fault set as defined by the adopted fault model. One possible approach is to start from a random set of test patterns. Fault simulation then determines how many of the potential faults are detected. With the obtained results as guidance extra vectors can be added or removed iteratively. An alternative and potentially more attractive approach relies on the knowledge of the functionality of a Boolean network to derive a suitable test vector for a given fault.

A fault simulator measures the quality of a test program. It determines the fault coverage which is defined as the total number of faults detected by the test sequence divided by two times the number of nodes in the network each node can give rise to a sa0 – sa1 fault. The obtained coverage is only as good as the fault models used.

Most common approach to fault simulation is the parallel fault simulation technique in which the correct circuit is simulated concurrently with a number of faulty ones each of which has a single fault injected. The results are compared, and a fault is labeled as detected for a given test vector set if the outputs diverge.

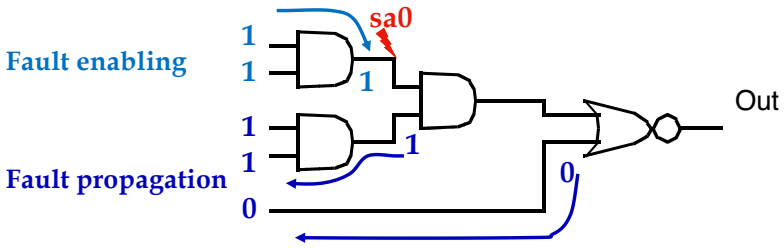
Automatic Test Pattern Generation (ATPG)

- Test pattern generation is tedious
- Automatic Test Pattern Generation (ATPG) tools produce a good set of vectors for each block of combinational logic
- Scan chains are used to control and observe the blocks
- Complete coverage requires a large number of vectors, raising the cost of test
- Most products settle for covering 90+% of potential stuck-at faults

Modern design flow now uses automated CAD tools to help solve the VLSI test problem. For example, you will be learning to use Cadence's MODUS package to help you make your design easily testable.

Path Sensitization

Goals: Determine input pattern that makes a fault controllable (triggers the fault, and makes its impact visible at the output nodes)



Techniques Used: D-algorithm, Podem

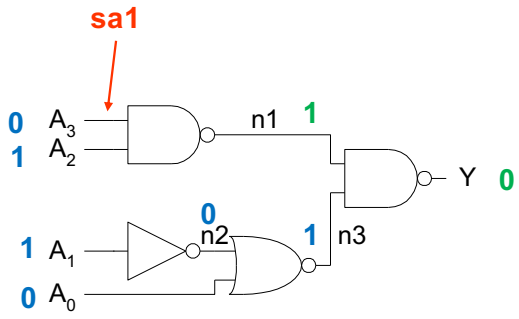
One common and straight forward way to generate test vector for a given circuit is to use linear-feedback shift register (LFSR) to generate pseudo-random binary sequence (PRBS) as input test vectors and use simulation to produce the expected output. This was the object of the Lab 4 exercise.

An alternative to random pattern generation and potentially more attractive approach for test pattern generation **relies on the knowledge of the functionality of a Boolean network** to derive a suitable test vector for a given fault.

Consider the above circuit. The goal is to determine the input excitation that exposes a sa0 fault occurring at the indicated node at the output of the network. The first requirement of such an excitation is that it should force the fault to occur (controllability). In this case we are looking for a pattern to force the node to 1 under normal circumstances. The only option here is to have {1,1} inputs in the top AND gate. Next the faulty signal has to propagate to the output to be observed. This phase is called **path sensitizing**. For any change in the faulty node to propagate it is necessary for the second input to the second level AND to be {1} and the 5^{th} input to be {0}. The test vector can now be assembled: {1,1,1,1,0}. The derivation of a minimum test vector set for more complex circuits is substantially more complex.

Example to generate minimum test vector set

	SA1	SA0
A_3	{0110}	{1110}
A_2	{1010}	{1110}
A_1	{0100}	{0110}
A_0	{0110}	{0111}
N1	{1110}	{0110}
N2	{0110}	{0100}
N3	{0101}	{0110}
Y	{0110}	{1110}



- Minimum set: {0100, 0101, 0110, 0111, 1010, 1110}

The goal here is to thoroughly test this combinational circuit for stuck-at faults with 100% coverage. There are four input nodes, one output node and three internal nodes. Each of these could be stuck at 0 or 1.

To test for sa1 fault for A3, set this to 0 (shown in blue). To propagate this fault forward, A2=1, resulting in N1 =1 (shown in green). Now we propagate Y=0 backwards towards the inputs.

For ~1 = 1 be observable at output Y as 0, N3 must be 1. This implies that N2 and A0 must be 0. Finally, this results in A1 = 1.

This generates one test vector {0110} for {A3..A0} that will detect sa1 fault at A3. If we do the same for all the other nodes, we have a full set of test vectors that will detect ALL sa-faults in the circuit. However, some test vectors will detect more than one stuck-at fault. For example, the test vector {0110} will also detect sa-1 faults at A3, A0, N2 and Y, and sa-0 faults at A1, N1 and N3.

So, the minimum set of test vectors for this circuit is now reduced to 6 tests as shown.

In generally, this process can be automated using specially designed CAD tools such that the test vector can be generated automatically (ATPG tool).